

## Backend API & Architecture Understanding - Inventory Project

### 1. Dependency Injection in FastAPI:

How does `Depends(get_db)` work in our routers, and why do we use it instead of creating a new database connection inside each API function manually?

### 2. Role-Based Access Control (RBAC):

In our `users.py` router, we use `Depends(require_permission('Users', 'FULL'))`. Explain step-by-step how the `require_permission` dependency verifies if the logged-in user has the right access. Where does it get the users token from?

### 3. Database Transactions (Commit vs Flush):

In `products.py`, when a new product is created, we use `db.add(product)`, `db.flush()`, and later `db.commit()`. What is the exact difference between `flush()` and `commit()`, and why is `flush()` necessary before creating the Inventory record for that product?

### 4. Data Validation and Schemas:

What role do Pydantic schemas (like `ProductCreate` or `UserOut` found in `schemas.py`) play in the API endpoints? How do they protect our database from bad or malicious data?

### 5. Separation of Concerns & Business Logic:

Why is the stock quantity adjusted by creating new `StockMovement` and `StockAdjustment` records in the `inventory.py` router, instead of just running a simple API to update the `quantity_on_hand` field directly in the Inventory table?

### 6. Password Hashing and Security:

In `auth.py`, when a user registers, we use `get_password_hash()`. Why is it crucial to hash passwords before storing them in the database instead of storing plain text?